



PRATHYUSHAENGINEERING COLLEGE

(AnAutonomous Institution)

Thiruvallur – 602025

***“GPU Architecture: From Graphics Processing to
General-Purpose Computing”***

CASE STUDY

OF

241GES301J-DIGITAL PRINCIPLE AND COMPUTER ORGANISATION

The Bachelor of Engineering (B.E)

IN

**B.E – CSE(ARTIFICIAL INTELLIGENCE AND MACHINE
LEARNING)**

Submitted by

POOSHITHA RAJESWARI S(111425148057)

Signature

**Mrs .A.PRIYADHARSHINI
ASSISTANT PROFESSOR**

1. Abstract

A Graphics Processing Unit (GPU) is a highly parallel processor that was originally developed for graphics and has evolved into a major accelerator for general-purpose computing. Modern GPUs contain many execution resources arranged into multiprocessors and use a hierarchy of registers, caches and high-bandwidth memory. This architecture allows many threads to execute concurrently, making GPUs useful for graphics, artificial intelligence, scientific computing, data processing, simulations and image processing. This case study explains GPU organization, execution, memory hierarchy, CPU–GPU cooperation, applications, advantages and limitations.

2. Introduction

GPU architecture is based on a different design goal from traditional CPU architecture. CPUs generally emphasize low-latency execution of a relatively small number of powerful threads, whereas GPUs are designed to achieve high throughput by executing large numbers of threads in parallel. NVIDIA's CUDA documentation describes this distinction and notes the use of GPUs across scientific computing, databases, analytics, image classification and generative AI.

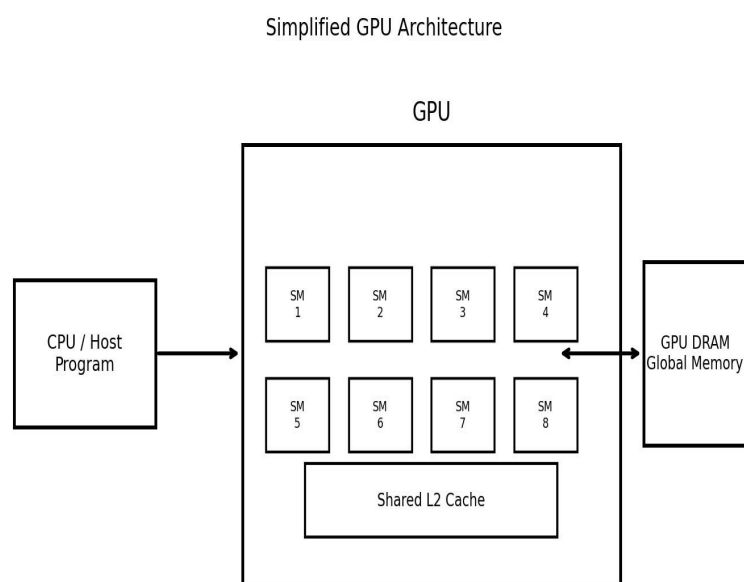


Figure 1. Original simplified GPU architecture diagram.

3. Objectives

- To understand the basic organization of a modern GPU.
- To study streaming multiprocessors and parallel execution.
- To understand the GPU memory hierarchy.
- To compare CPU and GPU design goals.
- To examine real-world applications and limitations
- GPU Architecture

A modern GPU can be viewed as many processing units connected through a memory hierarchy. NVIDIA describes a high-level architecture containing Streaming Multiprocessors (SMs), on-chip L2 cache and high-bandwidth DRAM. Arithmetic operations are executed by the SMs, while data and code move through the memory system.

3.1 Streaming Multiprocessors (SMs)

An SM is a major execution building block. It contains instruction scheduling resources, registers, cache/shared-memory resources and multiple functional units. Thread blocks are assigned to SMs, where their threads can execute concurrently.

3.2 Memory Hierarchy

Registers are private to threads, shared memory is available to threads in a block, caches reduce access latency, and global memory is backed by GPU-attached DRAM. The different levels trade speed, capacity and scope of access.

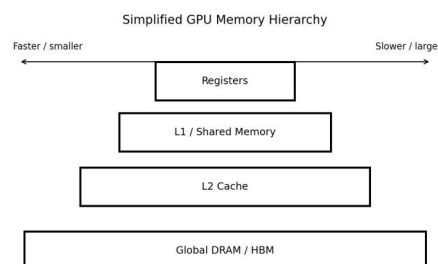


Figure 2. Simplified GPU memory hierarchy.

4. CPU vs GPU

CPUs are optimized for general-purpose control, branching, sequential work and low latency.

GPUs devote more resources to parallel data processing and can execute thousands of threads concurrently.

As a result, a GPU can be highly effective when the same operation is applied independently to many data elements.

CPU and GPU: Different Design Goals

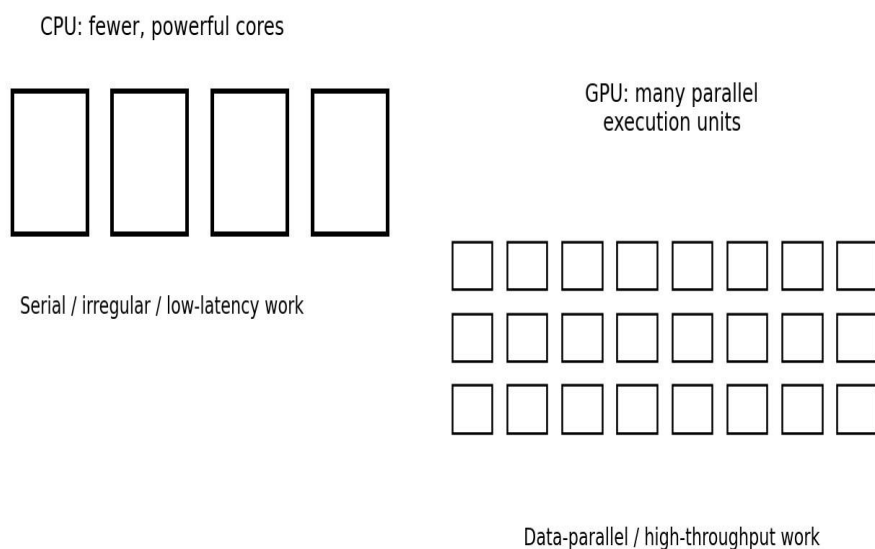


Figure 3. Conceptual comparison of CPU and GPU design goals

5. GPU Execution Model

In a heterogeneous system, the CPU acts as the host and the GPU as a compute device.

A program launches a kernel; the work is organized into grids, blocks and threads; and blocks are scheduled onto available SMs.

This hierarchy allows large numbers of lightweight threads to execute concurrently.

Typical CPU–GPU Execution Workflow

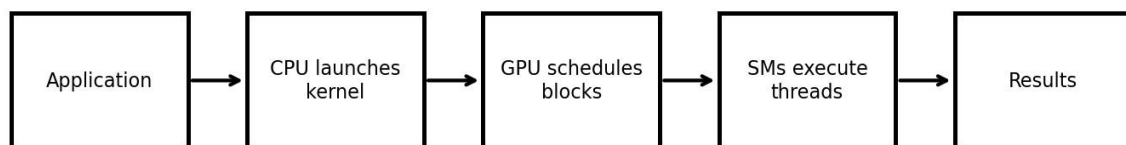


Figure 4. Typical CPU–GPU execution workflow.

6. Role of GPUs in Modern Computing

6.1 Artificial Intelligence and Machine Learning

Neural networks perform large numbers of matrix and vector operations. These operations often have substantial data parallelism, allowing GPU hardware to provide high throughput for training and inference. GPU computing is widely used in deep learning and high-performance computing.

6.2 Scientific and Engineering Computing

Simulations and numerical methods often repeat calculations over large arrays or grids. Such regular workloads can benefit from GPU parallelism.

6.3 Graphics and Gaming

Rendering requires large numbers of operations on pixels, vertices and textures. Parallel execution is therefore central to modern graphics processing.

6.4 Data Processing

Large-scale aggregations, joins, sorting and related operations can benefit when their work can be divided across many independent data elements. Recent GPU data-processing guidance highlights this use of GPU parallelism.

6.5 Image and Video Processing

Image and video algorithms frequently apply similar operations to many pixels or frames, making them suitable candidates for GPU acceleration.

7. Advantages

- Massive parallelism for suitable workloads.
- High throughput and memory bandwidth.
- Strong performance for large arrays, matrices and images.
- Useful acceleration for AI, graphics and scientific workloads.

8. Limitations

- Not every algorithm has enough parallelism.
- CPU–GPU data transfers can introduce overhead.
- Irregular memory access and heavy branching can reduce efficiency.
- Effective GPU programming may require specialized optimization knowledge.

9. Key Findings

Aspect	Finding
Architecture	Many parallel execution resources organized into processing units such as SMs.
Memory	Registers, shared memory, caches and global memory provide different speed/capacity trade-offs.
Execution	Threads are organized into blocks and grids or analogous structures.
Applications	AI, graphics, scientific computing, analytics and image/video processing can benefit.
Performance	Results depend on parallelism, memory access, workload size and transfer overhead.

10. Conclusion

GPU architecture demonstrates how computer organization has evolved toward massive parallelism. By combining many execution resources with a multi-level memory hierarchy, GPUs can deliver high throughput on suitable workloads. They are now important in graphics, AI, scientific computing, data processing and image/video applications. CPUs and GPUs are complementary: CPUs provide general-purpose control and strong single-thread performance, while GPUs accelerate sections of applications that contain substantial parallelism.

11. References

1. NVIDIA, CUDA Programming Guide — GPU introduction, CPU/GPU design differences and applications.
2. NVIDIA, GPU Performance Background — architecture fundamentals, SMs, cache and DRAM
3. NVIDIA, CUDA Programming Model — execution and memory hierarchy.
4. NVIDIA, PTX ISA Documentation — thread hierarchy and machine model.
5. Intel, oneAPI GPU Optimization Guide — general-purpose GPU execution model.
6. **Figure note:** All figures in this report are original simplified diagrams created for educational use; they are not vendor-specific chip floorplans.